

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

def load_and_prepare_data(filepath):
    """
    Loads data and handles column renaming based on the specific Kaggle dataset.
    """
    try:
        # Attempt to read CSV (handling common encoding issues with text data)
        df = pd.read_csv(filepath, encoding='latin-1')

        # The specific dataset (tmehul/spamcsv) usually has 'Category' and 'Message'
        # Or 'v1' and 'v2'. We standardize them here.
        if 'v1' in df.columns:
            df = df.rename(columns={'v1': 'label', 'v2': 'message'})
        elif 'Category' in df.columns:
            df = df.rename(columns={'Category': 'label', 'Message': 'message'})

        # Keep only necessary columns
        df = df[['label', 'message']]

        # Convert labels to binary numbers for the model (Spam=1, Ham=0)
        # This helps in calculating the Confusion Matrix later
        df['label_num'] = df['label'].map({'ham': 0, 'spam': 1})

        return df
    except FileNotFoundError:
        print("Error: The file 'spam.csv' was not found.")
        return None

def main():
    # 1. DATASET loading
    # Download the dataset from the link in your image and save as 'spam.csv'
    df = load_and_prepare_data('spam.csv')

    if df is None:
        return

    print(f"Dataset loaded. Total emails: {len(df)}")
    print(df.head())
    print("-" * 30)

    # 2. METHODOLOGY: Split Data
    # We split into Training (80%) and Testing (20%) sets
    X_train, X_test, y_train, y_test = train_test_split(
        df['message'],
        df['label_num'],
        test_size=0.2,
        random_state=42
    )

    # 3. METHODOLOGY: Feature Extraction & Preprocessing
    # CountVectorizer handles:
    # - Tokenization (breaking text into words)
    # - Stop-word removal (removing 'the', 'is', 'a', etc.)
    # - Word Frequencies (creating the Bag of Words model)
    vectorizer = CountVectorizer(stop_words='english')

    # Fit the vectorizer to training data and transform it
    X_train_count = vectorizer.fit_transform(X_train)

    # Transform the test data (do not fit again, use training vocabulary)
    X_test_count = vectorizer.transform(X_test)

    # 4. ALGORITHM: Naïve Bayes Classifier
    # MultinomialNB is standard for integer word counts
    model = MultinomialNB()
    model.fit(X_train_count, y_train)

    # 5. CLASSIFICATION (Using MAP decision rule implicitly)
    y_pred = model.predict(X_test_count)

    # 6. EVALUATION
    accuracy = accuracy_score(y_test, y_pred)
    print(f"Model Accuracy: {accuracy:.2%}")
    print("\nClassification Report:")
    print(classification_report(y_test, y_pred, target_names=['Ham', 'Spam']))

```

```

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

# Optional: Test with a custom email
print("-" * 30)
print("Testing custom examples:")
emails = [
    "Congratulations! You've won a $1,000 Walmart gift card. Click here to claim.",
    "Hey, are we still meeting for lunch tomorrow?"
]
emails_count = vectorizer.transform(emails)
predictions = model.predict(emails_count)

for email, pred in zip(emails, predictions):
    label = "Spam" if pred == 1 else "Ham"
    print(f"Email: '{email}' \n -> Classified as: {label}\n")

if __name__ == "__main__":
    main()

```

Dataset loaded. Total emails: 5572

	label	message	label_num
0	ham	Go until jurong point, crazy.. Available only ...	0
1	ham	Ok lar... Joking wif u oni...	0
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...	1
3	ham	U dun say so early hor... U c already then say...	0
4	ham	Nah I don't think he goes to usf, he lives aro...	0

Model Accuracy: 98.39%

Classification Report:

	precision	recall	f1-score	support
Ham	0.99	0.99	0.99	965
Spam	0.96	0.92	0.94	150
accuracy			0.98	1115
macro avg	0.97	0.96	0.96	1115
weighted avg	0.98	0.98	0.98	1115

Confusion Matrix:

```

[[959  6]
 [ 12 138]]

```

Testing custom examples:

Email: 'Congratulations! You've won a \$1,000 Walmart gift card. Click here to claim.'
-> Classified as: Spam

Email: 'Hey, are we still meeting for lunch tomorrow?'
-> Classified as: Ham